```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from google.colab import files
uploaded = files.upload()
```

Choose files 1_boston_housing.csv

1_boston_housing.csv(text/csv) - 35735 bytes, last modified: 06/05/2026 - 100% done

Saving 1_boston_housing.csv to 1_boston_housing.csv

```
df = pd.read_csv("1_boston_housing.csv")
df.head()
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2

Next steps: Generate code with df | New interactive sheet

```
from sklearn.model_selection import train_test_split

X = df.loc[:, df.columns != 'MEDV']
y = df.loc[:, df.columns == 'MEDV']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=123)
```

```
from sklearn.preprocessing import MinMaxScaler
mms = MinMaxScaler()
mms.fit(X_train)
X_train = mms.transform(X_train)
X_test = mms.transform(X_test)
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()

model.add(Dense(128, input_shape=(13, ), activation='relu', name='dense_1'))
model.add(Dense(64, activation='relu', name='dense_2'))
model.add(Dense(1, activation='linear', name='dense_output'))

model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 128)	1,792
dense_2 (Dense)	(None, 64)	8,256
dense_output (Dense)	(None, 1)	65

Total params: 10,113 (39.50 KB)
Trainable params: 10,113 (39.50 KB)
Non-trainable params: 0 (0.00 B)

```
history = model.fit(X_train, y_train, epochs=100, validation_split=0.05, verbose = 1)
```

```
Epoch 1/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - loss: 574.0303 - mae: 22.0700 - val_loss: 585.4668 - val_mae: 22.2100
Epoch 2/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 519.6892 - mae: 20.7303 - val_loss: 514.4974 - val_mae: 20.4879
Epoch 3/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 437.3260 - mae: 18.4779 - val_loss: 409.2673 - val_mae: 17.5897
Epoch 4/100
```

```
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 321.0310 - mae: 15.0583 - val_loss: 278.6911 - val_mae: 13.4099
Epoch 5/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 204.4280 - mae: 11.3644 - val_loss: 169.0209 - val_mae: 9.2642
Epoch 6/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 142.3054 - mae: 9.4002 - val_loss: 130.6999 - val_mae: 8.4013
Epoch 7/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 126.3685 - mae: 8.7645 - val_loss: 117.5661 - val_mae: 7.9526
Epoch 8/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 107.8245 - mae: 7.9437 - val_loss: 108.6239 - val_mae: 7.4831
Epoch 9/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 91.6416 - mae: 7.1154 - val_loss: 98.7619 - val_mae: 7.0138
Epoch 10/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 79.1879 - mae: 6.5197 - val_loss: 89.9395 - val_mae: 6.6236
Epoch 11/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 69.9419 - mae: 6.0652 - val_loss: 82.6240 - val_mae: 6.3655
Epoch 12/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 62.2740 - mae: 5.6751 - val_loss: 77.8095 - val_mae: 6.1696
Epoch 13/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 57.1047 - mae: 5.3336 - val_loss: 74.8139 - val_mae: 5.9457
Epoch 14/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 52.9523 - mae: 5.1195 - val_loss: 70.7419 - val_mae: 6.0751
Epoch 15/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 50.2431 - mae: 5.0026 - val_loss: 68.4628 - val_mae: 5.9291
Epoch 16/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 47.6379 - mae: 4.7744 - val_loss: 67.2563 - val_mae: 5.7009
Epoch 17/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 45.2183 - mae: 4.6737 - val_loss: 64.1165 - val_mae: 5.7957
Epoch 18/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 44.1746 - mae: 4.7590 - val_loss: 61.4800 - val_mae: 5.8170
Epoch 19/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 41.2258 - mae: 4.4397 - val_loss: 62.0612 - val_mae: 5.3364
Epoch 20/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 39.2652 - mae: 4.3176 - val_loss: 57.7145 - val_mae: 5.5487
Epoch 21/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 37.2617 - mae: 4.2477 - val_loss: 57.1020 - val_mae: 5.2726
Epoch 22/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 35.3910 - mae: 4.0908 - val_loss: 54.9015 - val_mae: 5.2225
Epoch 23/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 33.8476 - mae: 4.0510 - val_loss: 52.2638 - val_mae: 5.2071
Epoch 24/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 32.0568 - mae: 3.8920 - val_loss: 51.8740 - val_mae: 4.9960
Epoch 25/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 30.6059 - mae: 3.8634 - val_loss: 49.2090 - val_mae: 4.9849
Epoch 26/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 28.8827 - mae: 3.7138 - val_loss: 48.9204 - val_mae: 4.7785
Epoch 27/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 27.4463 - mae: 3.5691 - val_loss: 47.1405 - val_mae: 4.7571
Epoch 28/100
11/11 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 26.5603 - mae: 3.6317 - val_loss: 45.0140 - val_mae: 4.7393
Epoch 29/100
```

```
mse_nn, mae_nn = model.evaluate(X_test, y_test)

print('Mean squared error on test data: ', mse_nn)
print('Mean absolute error on test data: ', mae_nn)
```

```
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 22.9441 - mae: 3.1258
Mean squared error on test data:  22.944067001342773
Mean absolute error on test data:  3.125803232192993
```